

12 | 智能指针：从所有权看智能指针

唐刚 · Rust 语言从入门到实战



你好，我是 Mike。从今天开始，我们进入 Rust 进阶篇。

相对于基础篇，进阶篇更像外功招式，主要是掌握一些实用的基础设施，提高编程效率。这节课我们就在所有权视角下来学习 Rust 中的智能指针。

智能指针

学习智能指针之前，我们先来了解一下指针是什么。

指针和指针的类型

如果一个变量，里面存的是另一个变量在内存里的地址值，那么这个变量就被叫做**指针**。而我们前面讲到的引用（用 & 号表示）就是一种指针。

引用是必定有效的指针，它一定指向一个目前有效（比如没有被释放掉）的类型实例。而指针不一定是引用。也就是说，在 Rust 中，还有一些其他类型的指针存在，我们这节课就来学习

其中一些。

我们这里要再次明晰一下**引用的类型**。引用分为不同的类型，单独的 & 符号本身没有什么意义，但是它和其他类型组合起来就能形成各种各样的引用类型。比如：

&str 是字符串切片引用类型。

&String 是所有权字符串的引用类型。

&u32 是 u32 的引用类型。

注：&str、&String、&u32 都是一个整体。

这三种都是引用类型，作为引用类型，它们之间是不同的。但是同一种引用类型的实例，比如 &10u32 和 &20u32，它们的类型是相同的。

那么，指针其实也类似，指向不同类型实例的指针，它的类型也是有区别的，这叫做**指针的类型**。

智能指针

Rust 中指针的概念非常灵活，比如，它可以是一个结构体类型，只要其中的一个字段存储其他类型实例的地址，然后对这个结构体实现一些 Rust 标准库里提供的 trait，就可以把它变成指针类型。这种指针可以在传统指针的基础上添加一些额外信息，比如放在额外的一些字段中；也可以做一些额外操作，比如管理引用计数，资源自动回收等。从而显得更加智能，所以被叫做**智能指针**。

其实，我们前面碰到的 String 和 Vec<T> 就是一种智能指针。我们来看标准库代码中 [String](#) 的定义和 [Vec<T>](#) 的定义。

 复制代码

```
1 pub struct String {  
2     vec: Vec<u8>,  
3 }
```

 复制代码

```
1 pub struct Vec<T, #[unstable(feature = "allocator_api", issue = "32838")] A: Allo
2     buf: RawVec<T, A>,
3     len: usize,
4 }
```

通过代码我们可以看到，String 和 Vec<T> 实际都定义为结构体。

注：Rust 中智能指针的概念也直接来自于 C++。C++ 里面有 [unique_ptr](#)、[shared_ptr](#)。

智能指针可以让代码的开发相对来说容易一些。经过前面的学习，我们知道 Rust 基于所有权出发，定义了一套完整的所有权和借用规则。很多我们习以为常的代码写法，在 Rust 中变成了“违法”，这导致很多人觉得学习 Rust 的门槛很高。而智能指针可以在某些方面降低这种门槛。

它如何能做到呢？我们稍后揭秘。

前面我们看到过这种代码：

 复制代码

```
1 fn foo() -> u32 {
2     let i = 100u32;
3     i
4 }
5 fn main() {
6     let _i = foo();
7 }
```

我们看到，foo() 函数将 i 返回用的不是 move 行为，而是 copy 行为，将 100u32 这个值复制了一份，返回给外面的 _i。foo() 函数调用结束后，foo() 里的局部变量 i 被销毁。

我们再回忆一下另一段代码。

 复制代码

```
1 fn foo() -> String {
2     let s = "abc".to_string();
3     s
4 }
5 fn main() {
6     let _s = foo();
7 }
```

上述代码可以在函数 `foo()` 里生成一个字符串实例，这个字符串实例资源在堆内存中分配，`s` 是 `foo` 函数里的局部变量，拥有字符串资源的所有权。在代码的最后一行，`s` 把所有权返回给外部调用者并传递给 `_s`。`foo()` 调用完成后，栈上的局部变量 `s` 被销毁。

这种写法可行是因为返回了资源的所有权。如果我们把代码里的 `String` 换成 `&String`，把 `s` 换成 `&s` 就不行了。

 复制代码

```
1 fn foo() -> &String {
2     let s = "abc".to_string();
3     &s
4 }
5 fn main() {
6     let _s = foo();
7 }
```

你可能会问，既然 `String` 资源本身是在堆中，为什么我们不能拿到这个资源的引用而返回呢？

我们来看看为什么不行。在 `foo()` 函数里，其实我们返回的并不是那个堆里字符串资源的引用，而是栈上局部变量 `s` 的引用。堆里的字符串资源由栈上的变量 `s` 管理，而 `s` 在 `foo()` 函数调用完成后，就被销毁了，堆里的字符串资源也一并被回收了，所以刚刚那段代码当然行不通了。

同样的，下面这段代码也是不允许的。

 复制代码

```
1 fn foo() -> &u32 {  
2     let i = 100u32;  
3     &i  
4 }  
5 fn main() {  
6     let _i = foo();  
7 }
```

那么，我们有什么办法能让这种意图变得可行呢？其实是有的，比如使用 `Box<T>` 智能指针。

`Box<T>`

`Box<T>` 是一个类型整体，作为智能指针 `Box<T>` 可以把资源强行创建在堆上，并获得资源的所有权，让资源的生命期得以被程序员精确地控制。

注：堆上的资源，默认与整个程序进程的存在时间一样久。

我们来看使用 `Box<T>` 如何处理前面那个示例。

 复制代码

```
1 fn foo() -> Box<u32> {  
2     let i = 100u32;  
3     Box::new(i)  
4 }  
5 fn main() {  
6     let _i = foo();  
7 }
```

通过 `Box`，我们把栈上 `i` 的值，强行 `copy` 了一份并放在堆上某个地址，然后 `Box` 指针指向这个地址。

返回一个整数 `i` 的指针确实没多大用，如果我们定义了一个结构体，可以采用类似的办法从函数中返回结构体的 `Box` 指针。

```
1 struct Point {
2     x: u32,
3     y: u32
4 }
5
6 fn foo() -> Box<Point> {
7     let p = Point {x: 10, y: 20}; // 这个结构体的实例创建在栈上
8     Box::new(p)
9 }
10 fn main() {
11     let _p = foo();
12 }
```

这就很有用了。

我们看示例的第 7 行，Point 的实例 p 实际是创建在栈上的。通过 `Box::new(p)`，把 p 实例强行按位复制了一份，并且放到了堆上，我们记为 p'。然后 `foo()` 函数返回，把 Box 指针实例 move 给了 _p。之后，_p 拥有了对 p' 的所有权。

Box<T> 中的所有权分析

我们继续深入，回顾一下上面示例里讲到的，编译期间已知尺寸的类型实例会默认创建在栈上。Point 有两个字段：x、y，它们的尺寸是固定的，都是 4 个字节，所以 Point 的尺寸就是 8 个字节，它的尺寸也是固定的。所以它的实例会被创建在栈上。第 7 行的 p 拥有这个 Point 实例的所有权。注意 Point 并没有默认实现 Copy，虽然它的尺寸是固定的。

在创建 `Box<Point>` 实例的时候会发生所有权转移：资源从栈上 move 到了堆上，原来栈上的那片资源被置为无效状态，因此下面的代码编译不会通过。

```
1 struct Point {
2     x: u32,
3     y: u32
4 }
5
6 fn foo() -> Box<Point> {
7     let p = Point {x: 10, y: 20};
```

```

8     let boxed = Box::new(p); // 创建Box实例
9     let q = p;               // 这一句用来检查p有没有被move走
10    boxed
11 }
12 fn main() {
13     let _p = foo();
14 }

```

编译提示：

 复制代码

```

1 error[E0382]: use of moved value: `p`
2 --> src/main.rs:9:13
3 |
4 7 |     let p = Point {x: 10, y: 20};
5 |         - move occurs because `p` has type `Point`, which does not implement
6 8 |     let boxed = Box::new(p); // 创建Box实例
7 |                   - value moved here
8 9 |     let q = p; // 这一句用来检查p有没有被move走
9 |         ^ value used here after move

```

之所以会发生所有权这样的转移，是因为 Point 类型本身就是 move 语义的。作为对照，我们来看一个示例。

 复制代码

```

1 fn foo() -> Box<u8> {
2     let i = 5;
3     let boxed = Box::new(i); // 创建Box实例
4     let q = i;               // 这一句用来检查i有没有被move走
5     boxed
6 }
7
8 fn main() {
9     let _i = foo();
10 }

```

这个示例就可以编译通过。也就是说，在执行 `Box::new()` 创建 Box 实例时，具有 copy 语义的整数类型和具有 move 语义的 Point 类型行为不一样。整数会 copy 一份自己，Point 实例会把自己 move 到 Box 里面去。

一旦创建好 Box 实例后，这个实例就具有了对里面资源的所有权了，它是 move 语义的，你可以看一下示例。

 复制代码

```
1 fn foo() -> Box<u8> {
2     let i = 5;
3     let boxed = Box::new(i);    // 创建Box实例
4     let q = i;                  // 这一句用来检查i有没有被move走
5     let boxed2 = boxed;         // 这一句检查boxed实例是不是move语义
6     boxed
7 }
8
9 fn main() {
10     let _i = foo();
11 }
```

不能编译通过，提示：

 复制代码

```
1 error[E0382]: use of moved value: `boxed`
2 --> src/main.rs:6:5
3 |
4 3 |     let boxed = Box::new(i);    // 创建Box实例
5 |         ----- move occurs because `boxed` has type `Box<u8>`, which does not
6 4 |     let q = i;                  // 这一句用来检查i有没有被move走
7 5 |     let boxed2 = boxed;         // 这一句检查boxed实例是不是move语义
8 |         ----- value moved here
9 6 |     boxed
10 |     ^^^^^ value used hereafter move
```

这个示例就验证了我们刚才的说法。

Box<T> 的解引用

前面我们讲过，创建一个 Box 实例把栈上的内容包起来，可以把栈上的值移动到堆上，比如：

 复制代码

```
1 let val: u8 = 5;
2 let boxed: Box<u8> = Box::new(val); // 这里 boxed 里面那个u8就是堆上的值
```

还可以在 Box 实例上使用**解引用符号 ***，把里面的堆上的值再次移动回栈上，比如：

```
1 let boxed: Box<u8> = Box::new(5);
2 let val: u8 = *boxed; // 这里这个val整数实例就是在栈上的值
```

[复制代码](#)

解引用是 `Box::new()` 的**逆操作**，可以看到整个过程是相反的。

对于具有 copy 语义的 u8 类型来说，解引用回来后，boxed 还能使用，我们看下示例。

```
1 fn main() {
2     let boxed: Box<u8> = Box::new(5);
3     let val: u8 = *boxed;
4
5     println!("{:?}", val);
6     println!("{:?}", boxed); // 用于u8类型，解引用后，boxed实例还能用
7 }
8 // 输出
9 5
10 5
```

[复制代码](#)

而对于具有 move 语义的类型来说，情况就不一样了，会发生所有权的转移。比如：

```
1 #[derive(Debug)]
2 struct Point {
3     x: u32,
4     y: u32
5 }
6
7 fn main() {
8     let p = Point {x: 10, y: 20};
9     let boxed: Box<Point> = Box::new(p);
10    let val: Point = *boxed; // 这里做了解引用，Point实例回到栈上
```

[复制代码](#)

```

11
12     println!("{:?}", val);
13     println!("{:?}", boxed); // 解引用后想把boxed再打印出来
14 }

```

编译出错，提示 `*boxed` 已经 move 了。

[复制代码](#)

```

1 error[E0382]: borrow of moved value: `boxed`
2   --> src/main.rs:13:22
3   |
4 10 |         let val: Point = *boxed;
5   |                             ----- value moved here
6   ...
7 13 |         println!("{:?}", boxed);
8   |                             ^^^^^^ value borrowed here after move

```

因此 boxed 不能再使用。也就是说，如果 `Box<T>` 的 T 是 move 语义的，那么对这个 Box 实例做解引用操作，会把这个 Box 实例的所有权释放。

关于这些细节，其实你不用太担心能不能一次性掌握好，因为你用错的时候，Rustc 小助手会贴心地准确提示你，所以不要有心理负担。

`Box<T>` 实现了 trait

`Box<T>` 的好处在于它的明确性，它里面的资源一定在堆上，所以我们就不用再去关心资源是在栈上还是堆上这种细节问题了。一种类型，被 `Box<>` 包起来的过程就叫作这个类型的**盒化** (boxed)。

Rust 在标准库里为 `Box<T>` 实现了 `Deref`、`Drop`、`AsRef<T>` 等 trait，所以 `Box<T>` 可以直接调用 T 实例的方法，访问 T 实例的值。

[复制代码](#)

```

1 #[derive(Debug)]
2 struct Point {
3     x: u32,

```

```

4     y: u32,
5 }
6
7 impl Point {
8     fn play(&self) {
9         println!("I'am a method of Point.");
10    }
11 }
12
13 fn main() {
14     let boxed: Box<Point> = Box::new(Point{x: 10, y: 20});
15     boxed.play();    // 点操作符触发deref
16     println!("{:?}", boxed);
17 }
18 // 输出
19 I'am a method of Point.
20 Point { x: 10, y: 20 }

```

`Box<T>` 拥有对 `T` 实例的所有权，所以可以对 `T` 实例进行写操作。

[复制代码](#)

```

1 #[derive(Debug)]
2 struct Point {
3     x: u32,
4     y: u32,
5 }
6
7 fn main() {
8     let mut boxed: Box<Point> = Box::new(Point{x: 10, y: 20});
9     *boxed = Point {    // 这一行，使用解引用操作更新值
10         x: 100,
11         y: 200
12     };
13     println!("{:?}", boxed);
14 }
15 // 输出
16 Point { x: 100, y: 200 }

```

`Box<T>` 的 Clone

`Box<T>` 能否 Clone，需要看 `T` 是否实现了 Clone，因为我们也需要把 `T` 的资源克隆一份。你可以看一下我给出的示例。

```
1 #[derive(Debug, Clone)]
2 struct Point {
3     x: u32,
4     y: u32,
5 }
6
7 impl Point {
8     fn play(&self) {
9         println!("I'am a method of Point.");
10    }
11 }
12
13 fn main() {
14     let mut boxed: Box<Point> = Box::new(Point{x: 10, y: 20});
15     let mut another_boxed = boxed.clone(); // 克隆
16     *another_boxed = Point{x: 100, y: 200}; // 修改新的一份值
17     println!("{:?}", boxed); // 打印原来一份值
18     println!("{:?}", another_boxed); // 打印新的一份值
19 }
20 // 输出
21 Point { x: 10, y: 20 }
22 Point { x: 100, y: 200 }
```

Box<T> 作为函数参数

我们可以把 `Box<T>` 作为参数传入函数，这个我们前面已经见过了。


```
1 #[derive(Debug)]
2 struct Point {
3     x: u32,
4     y: u32,
5 }
6
7 fn foo(p: Box<Point>) {    // 这里参数类型是 Box<Point>
8     println!("{:?}", p);
9 }
10
11 fn main() {
12     foo(Box::new(Point {x: 10, y: 20}));
13 }
14 // 输出
15 Point { x: 10, y: 20 }
```

&Box<T>

Box<T> 本身作为一种类型，对它做引用操作当然是可以的。

```
1 #[derive(Debug)]
2 struct Point {
3     x: u32,
4     y: u32,
5 }
6
7 impl Point {
8     fn play(&self) {
9         println!("I'am a method of Point.");
10     }
11 }
12
13 fn main() {
14     let boxed: Box<Point> = Box::new(Point{x: 10, y: 20});
15     boxed.play();          // 调用类型方法
16     let y = &boxed;        // 取boxed实例的引用
17     y.play();              // 调用类型方法
18     println!("{:?}", y);
19 }
20 // 输出
21 I'am a method of Point.
22 I'am a method of Point.
```

```
23 Point { x: 10, y: 20 }
```

在示例中，boxed 是一个所有权型变量，y 是一个引用型变量。它们都能调用到 Point 类型上的方法。

对 Box 实例做可变引用 (&mut) 也是可以的，你可以看一下示例。

[复制代码](#)

```
1  #[derive(Debug)]
2  struct Point {
3      x: u32,
4      y: u32,
5  }
6
7  impl Point {
8      fn play(&self) {
9          println!("I'am a method of Point.");
10     }
11 }
12
13 fn main() {
14     let mut boxed: Box<Point> = Box::new(Point{x: 10, y: 20});
15     let y = &mut boxed;      // 这里&mut Box<Point>
16     y.play();                // 调用类型方法
17     println!("{:?}", y);    // 修改前的值
18     **y = Point {x: 100, y: 200}; // 注意这里用了二级解引用
19     println!("{:?}", y);    // 修改后的值
20 }
21 // 输出
22 I'am a method of Point.
23 Point { x: 10, y: 20 }
24 Point { x: 100, y: 200 }
```

这个示例里值得注意的是第 18 行，做了两次解引用，第一次是对 &mut 做的，第二次是对 Box<T> 做的。

Box<Self>

前面我们讲过，类型的方法可以用 self、&self、&mut self 三种形态传入 Self 参数。其中第一种 self 形态还有一种变体 Box<Self>，你可以看一下示例。

```
1 #[derive(Debug)]
2 struct Point {
3     x: u32,
4     y: u32,
5 }
6
7 impl Point {
8     fn play_ref(&self) {
9         println!("I'am play_ref of Point.");
10    }
11    fn play_mutref(&mut self) {
12        println!("I'am play_mutref of Point.");
13    }
14    fn play_own(self) {
15        println!("I'am play_own of Point.");
16    }
17    fn play_boxown(self: Box<Self>) {    // 注意这里
18        println!("I'am play_boxown of Point.");
19    }
20 }
21
22 fn main() {
23     let mut boxed: Box<Point> = Box::new(Point{x: 10, y: 20});
24     boxed.play_ref();
25     boxed.play_mutref();
26     boxed.play_boxown();
27     // boxed.play_own();    // play_boxown()和 play_own() 只能同时打开一个
28 }
```

注意示例中，`play_boxown()` 和 `play_own()` 只能同时打开一个，这是为什么呢？你思考一下。

结构体中的 Box

`Box<T>` 作为类型，当然是可以出现在 `struct` 里的，你可以看一下示例。

```
1 struct Point {
2     x: u32,
3     y: u32,
4 }
5 struct Triangle {
```

```

6     one: Box<Point>,    // 三个字段类型都是 Box<Point>
7     two: Box<Point>,
8     three: Box<Point>,
9 }
10 fn main() {
11     let t = Triangle {
12         one: Box::new(Point {
13             x: 10,
14             y: 10,
15         }),
16         two: Box::new(Point {
17             x: 20,
18             y: 20,
19         }),
20         three: Box::new(Point {
21             x: 10,
22             y: 20,
23         }),
24     };
25 }

```

Box<dyn trait>

回忆 [第 10 讲](#) 的 trait object，它代表一种类型，这种类型可以代理一批其他的类型。但是 dyn trait 本身的尺寸在编译期是未知的，所以 dyn trait 的出现总是要借助于引用或智能指针。而 Box<dyn trait> 是最常见的，甚至比 &dyn trait 更常见。原因就是 Box<dyn Trait> 拥有所有权，这就是 Box<T> 方便的地方，而 &dyn Trait 不拥有所有权，有的时候就没那么方便。

我们来看使用 Box<dyn trait> 做函数参数的一个示例。

 复制代码

```

1 struct Atype;
2 struct Btype;
3 struct Ctype;
4
5 trait TraitA {}
6
7 impl TraitA for Atype {}
8 impl TraitA for Btype {}
9 impl TraitA for Ctype {}
10

```

```

11 fn doit(x: Box<dyn TraitA>) {}
12
13 fn main() {
14     let a = Atype;
15     doit(Box::new(a));
16     let b = Btype;
17     doit(Box::new(b));
18     let c = Ctype;
19     doit(Box::new(c));
20 }

```

这个示例里的 `doit()` 函数能接收 `Atype`、`Btype`、`Ctype` 三种不同类型的实例。

如果 `dyn trait` 出现在结构体里，那么 `Box<dyn trait>` 形式就比 `&dyn trait` 形式要方便得多。比如，下面示例里的结构体字段类型是 `Box<dyn TraitA>`，能正常编译。

[复制代码](#)

```

1 struct Atype;
2 struct Btype;
3 struct Ctype;
4
5 trait TraitA {}
6
7 impl TraitA for Atype {}
8 impl TraitA for Btype {}
9 impl TraitA for Ctype {}
10
11 struct MyStruct {
12     x: Box<dyn TraitA> // 结构体的字段类型是 Box<dyn TraitA>
13 }
14
15 fn main() {
16     let a = Atype;
17     let t1 = MyStruct {x: Box::new(a)};
18     let b = Btype;
19     let t2 = MyStruct {x: Box::new(b)};
20     let c = Ctype;
21     let t3 = MyStruct {x: Box::new(c)};
22 }

```

而下面这个示例，结构体字段类型是 `&dyn TraitA`，就没办法通过编译。

```

1 struct Atype;
2 struct Btype;
3 struct Ctype;
4
5 trait TraitA {}
6
7 impl TraitA for Atype {}
8 impl TraitA for Btype {}
9 impl TraitA for Ctype {}
10
11 struct MyStruct {
12     x: &dyn TraitA    // 结构体字段类型是 &dyn TraitA
13 }

```

报错如下：

```

1 error[E0106]: missing lifetime specifier
2   --> src/lib.rs:12:8
3   |
4 12 |     x: &dyn TraitA
5   |         ^ expected named lifetime parameter
6   |
7 help: consider introducing a named lifetime parameter
8   |
9 11 ~ struct MyStruct<'a> {
10 12 ~     x: &'a dyn TraitA
11   |

```

这个错误涉及到第 20 讲引用的生命期的概念，现在我们去深究。

`Box<T>` 智能指针的内容就讲到这里，下面我们看另一种智能指针 `Arc<T>`。

Arc<T>

`Box<T>` 是单所有权或独占所有权模型的智能指针，而 `Arc<T>` 是共享所有权模型的智能指针，也就是多个变量可以同时拥有一个资源的所有权。和 `Box<T>` 一样，`Arc<T>` 也会保证被包装的内容被分配在堆上。

clone

Arc 的主要功能是和 `clone()` 配合使用。在 Arc 实例上每一次新的 `clone()` 操作，总是会将资源的引用数 +1，而保持原来那一份资源不动，这个信息记录在 Arc 实例里面。每一个指向同一个资源的 Arc 实例走出作用域，就会给这个引用计数 -1。直到最后一个 Arc 实例消失，目标资源才会被销毁释放。你可以看一下示例。

[复制代码](#)

```
1 use std::sync::Arc;
2
3 #[derive(Debug)]    // 这里不需要目标type实现Clone trait
4 struct Point {
5     x: u32,
6     y: u32,
7 }
8
9 impl Point {
10     fn play(&self) {
11         println!("I'am a method of Point.");
12     }
13 }
14
15 fn main() {
16     let arced: Arc<Point> = Arc::new(Point{x: 10, y: 20});
17     let another_arced = arced.clone();    // 克隆引用
18     println!("{:?}", arced);             // 打印一份值
19     println!("{:?}", another_arced);     // 打印同一份值
20     arced.play();
21     another_arced.play();
22     let arc3_ref = &another_arced;
23     arc3_ref.play();
24 }
25 // 输出
26 Point { x: 10, y: 20 }
27 Point { x: 10, y: 20 }
28 I'am a method of Point.
29 I'am a method of Point.
30 I'am a method of Point.
```

我们可以看到，相比于 `Box<T>`，`Arc<T>` 的 clone 不要求 T 实现了 Clone trait。`Arc<T>` 的克隆行为只会改变 Arc 的引用计数，而不会克隆里面的内容。由于不需要克隆原始资源，所以性能是很高的。

类似于 `Box<T>`，`Arc<T>` 也实现了 `Deref`、`Drop`、`Clone` 等 trait。因此，`Arc<T>` 也可以符合人类的习惯，访问到里面类型 `T` 的方法。`Arc<T>` 的不可变引用 `&Arc<>` 也可以顺利调用到 `T` 上的方法。

Arc

和 `Box<T>` 一样，`Arc` 也可以用在方法中的 `self` 参数上面，作为所有权 `self` 的一个变体形式。

我们继续扩展上面的代码，你可以看一下扩展后的样子。

[复制代码](#)

```
1 use std::sync::Arc;
2
3 #[derive(Debug)]
4 struct Point {
5     x: u32,
6     y: u32,
7 }
8
9 impl Point {
10     fn play_ref(&self) {
11         println!("I'am play_ref of Point.");
12     }
13     fn play_mutref(&mut self) {
14         println!("I'am play_mutref of Point.");
15     }
16     fn play_own(self) {
17         println!("I'am play_own of Point.");
18     }
19     fn play_boxown(self: Box<Self>) { // 注意这里
20         println!("I'am play_boxown of Point.");
21     }
22     fn play_arcown(self: Arc<Self>) { // 注意这里
23         println!("I'am play_arcown of Point.");
24     }
25 }
26
27 fn main() {
28     let mut boxed: Box<Point> = Box::new(Point{x: 10, y: 20});
29     boxed.play_ref();
30     boxed.play_mutref();
31     boxed.play_boxown();
```



```
32 // boxed.play_own(); // play_boxown()和 play_own() 只能同时打开一个
33
34 let arced: Arc<Point> = Arc::new(Point{x: 10, y: 20});
35 arced.play_ref();
36 // arced.play_mutref(); // 不能用
37 // arced.play_own(); // 不能用, Arc<T> 中的T无法被移出
38 arced.play_arcown();
39 }
40 // 输出
41 I'am play_ref of Point.
42 I'am play_mutref of Point.
43 I'am play_boxown of Point.
44 I'am play_ref of Point.
45 I'am play_arcown of Point.
```

通过这个示例我们可以看到，不能通过 `Arc<>` 直接修改里面类型的值，也不能像 `Box<>` 的解引用操作那样，把里面的内容从 `Arc<>` 中移动出来。你可以试着打开示例里注释掉的几行看看 Rustc 小助手的提示信息。

Arc<dyn trait>

我们还可以把前面 `Box<dyn trait>` 的示例改编成 `Arc<T>` 的。

 复制代码

```
1 use std::sync::Arc;
2
3 struct Atype;
4 struct Btype;
5 struct Ctype;
6
7 trait TraitA {}
8
9 impl TraitA for Atype {}
10 impl TraitA for Btype {}
11 impl TraitA for Ctype {}
12
13 struct MyStruct {
14     x: Arc<dyn TraitA>
15 }
16
17 fn main() {
18     let a = Atype;
19     let t1 = MyStruct {x: Arc::new(a)};
```

```
20     let b = Btype;  
21     let t2 = MyStruct {x: Arc::new(b)};  
22     let c = Ctype;  
23     let t3 = MyStruct {x: Arc::new(c)};  
24 }
```

值的修改

多所有权条件下，怎么修改 Arc 里面的值呢？答案是不能修改。虽然 `Arc<T>` 是拥有所有权的，但 `Arc<T>` 不提供修改 T 的能力，这也是 `Arc<T>` 和 `Box<T>` 不一样的地方。后面我们在并发编程部分会讲到 `Mutex`、`RwLock` 等锁。想要修改 Arc 里面的内容，必须配合这些锁才能完成，比如 `Arc<Mutex<T>>`。

其实很好理解，共享所有权的场景下，如果任意一方能随意修改被包裹的值，那就会影响其他所有权的持有者，整个就乱套了。所以要修改的话必须引入锁的机制。

`Arc<T>` 与不可变引用 `&` 的区别

首先，它们都是共享对象的行为，本质上都是指针。但 `Arc<T>` 是共享了所有权模型，而 `&` 只是共享借用模型。共享借用模型就得遵循借用检查器的规则——借用的有效性依赖于被借用资源的 `scope`。对于这个的分析是非常复杂的。而所有权模型是由自己来管理资源的 `scope`，所以处理起来比较方便。

小结

这节课我们一起学习了最常用的两个智能指针：`Box<T>` 和 `Arc<T>`。其实 Rust 里还有很多智能指针，比如 `Rc`、`Cell`、`RefCell` 等等，每一种智能指针类型都有自己的特点。但是不管怎样，学习的方法都是一样的，那就是从所有权的视角去分析研究。你可以在后面遇到那些类型的时候，再根据这节课提供的方法去研究它们。

后面我们会看到，在智能指针的加持下，Rust 代码写起来会非常流畅，可以和 Java 不相上下。再结合 Rust 强大的类型系统建模能力，等你写得熟练之后，在中大项目中，使用 Rust 甚至会有超越 Python 的开发效率。

智能指针

定义

指针

指针的类型

智能的指针

Box<T>

强制资源在堆上

持有所有权

T 是 copy 的还是 move 的行为的区别

解引用

解引用中所有权的细节

Deref、Drop、AsRef

Clone

作为函数参数

Box 实例的引用

Box<Self> 作为方法的参数

结构体中的 Box<T>

Box<dyn Trait>

Arc<T>

引用计数智能指针

clone() 方法

Deref、Drop、Clone

Arc<Self>

Arc<dyn Trait>

不能修改里面的值

共享所有权与 & 共享引用的区别

思考题

你试着打开示例中的这两句，看看报错信息，然后分析一下是什么原因？

 复制代码

```
1 // arced.play_mutref(); // 不能用
2 // arced.play_own();    // 不能用
```

欢迎你把自己的分析分享到评论区，也欢迎你把这节课的内容分享给其他朋友，我们下节课再见！

© 版权归极客邦科技所有，未经许可不得传播售卖。页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

全部留言 (11)

最新 精选



Taozi

2023-11-15 来自上海

Arc本质上是个引用，所以不允许同时存在可变引用或者移动。

作者回复: 👍



 2



刘丹

2023-11-15 来自广东

在中大项目中，使用 Rust 甚至会有超越 Python 的开发效率。

作者回复: 对的，py代码量上来后，可维护性和可重构性比rust差得太远。



 2



asdf100

2023-11-30 来自澳大利亚

在执行 `Box::new()` 创建 `Box` 实例时，具有 `copy` 语义的整数类型和具有 `move` 语义的 `Point` 类型行为不一样。整数会 `copy` 一份自己，`Point` 实例会把自己 `move` 到 `Box` 里面去。

是否可以理解为如果创建的对象有`copy`语义的话，则进行`copy`；否则进行`move`操作？

作者回复：对的



1



Michael

2023-11-15 来自中国香港

`play_boxown()` 和 `play_own()` 只能同时打开一个，这两个方法调用都会消耗所有权，导致没法调用另外一个。

作者回复：👍



1



\$侯

2023-12-25 来自浙江

`Box<T>` 中的所有权分析 那里

说“`Point` 类型本身就是 `move` 语义的”

这里的`Point`中的元素不都是`copy`语义的吗，所以这个`Point`不是应该是`copy`语义的吗，为什么说成是`move`语义的

作者回复：这正是一个隐含的知识点，固定尺寸的结构体却默认不是可`copy`的。目的是为了防止不小心的产生额外复制负担。



superggn

2023-12-19 来自北京

`arcled.play_mutref()` => 这个不能用是因为 `Arc` 这种提供多个引用的类型不支持可变引用
`arcled.play_own()` => 这个不能用是因为 `Arc` 这种提供多个引用的类型不支持转移内部值的所有权

一开始我寻思这里会不会是直接抛出 `mismatched type`, 看 `rustc` 的提示发现不对, 后来想了想, 可能是因为 `rust` 找不到类型会先跑 `deref coercion`, `deref coercion` 匹配到了就不会报 `mismatched type`, 报错是在 `deref coercion` 过程中获取所有权的时候才出现?

总结下: `Box<T>`, `Arc<T>`, `Rc<T>` 调用 `T` 的方法, 机制都是 `deref coercion`

`Box` 还好, 所有权拿走就拿吧, 但 `Arc` / `Rc` 这两都可能会有不同的所有者, 里头的 `T` 不能是 `mut` 的, 所有权也不能放出去, 所以会错

作者回复: 分析得非常棒! 思路也很棒。



鸠摩智

2023-11-16 来自江苏

值得注意的是, 函数的入参是 `self:Box<Self>` 时, 调用这个函数的只能是 `Box` 包起来的类型, 比如这里的 `Point`。

```
let p = Point{x:1,y:2};
```

```
p.play_boxown();
```

会编译报错 `method `play_boxown` not found for this struct Point`

the method is available for `Box<Point>` here

consider wrapping the receiver expression with the appropriate type

```
Box::new(p).play_boxown();
```

点操作符不会自动用 `Box` 包起来, 但是会解开 `Box`。

作者回复: 对的, 👍。自动解开是因为 `Box<T>` 实现了到 `T` 的 `Deref`: <https://doc.rust-lang.org/std/boxed/struct.Box.html#impl-Deref-for-Box%3CT,+A%3E>



下雨天

2023-11-15 来自湖北

```
let arced: Arc<Point> = Arc::new(Point{x: 10, y: 20});
```

```
1. arced.play_mutref();
```

`arced` 只提供不可变引用, `play_mutref` 参数是可变引用。打开就会报错

```
2. arced.play_own(); // 不能用
```

play_own的参数self表示 Point , 而arced类型是Arc<Point> 不能强制转换。

作者回复: 👍。是的，里面的T无法被移出Arc，这点与Box不同。



-

2023-11-15 来自北京

play_boxown() 和 play_own() 只能同时打开一个,两个都是所有权转移了，所以就只能调用一次。

作者回复: 👍



duwoodly

2023-11-15 来自重庆

arced.play_mutref(); // Arc<T>没有实现智能指针的DerefMut trait
arced.play_own(); // 不能从Arc<T> 中移出值，除非T实现了Copy

作者回复: 对的，自己动手试验，得到最准确最全面的信息。 👍

